

UART Communication

Introduction to ARM Cortex-M Microprocessors – Chapter 12

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1** Why This Matters
- 2** Serial Fundamentals
- 3** Configuring the USART
- 4** UART as an Instrument
- 5** Design Challenge

- 1** Why This Matters
- 2 Serial Fundamentals
- 3 Configuring the USART
- 4 UART as an Instrument
- 5 Design Challenge

Until now, seeing what your program is doing has required a debugger and a breakpoint. The **UART** (Universal Asynchronous Receiver/Transmitter) changes that: with a few lines of code the microcontroller can print to a terminal on your PC. That makes it the single most valuable debugging tool in embedded development — and it is also how the device talks to GPS modules, Bluetooth radios, and industrial instruments. From this chapter onward, you should use it in every program you write.

After completing this chapter, you should be able to:

1. Contrast parallel with serial and synchronous with asynchronous transmission.
2. Describe every field of a UART frame and state the purpose of the start and stop bits.
3. Calculate the BRR register value for any required baud rate.
4. Configure a USART peripheral and its alternate-function pins.
5. Transmit and receive characters by polling and by interrupt.
6. Use UART output as a diagnostic instrument in your own programs.

- 1 Why This Matters
- 2 **Serial Fundamentals**
- 3 Configuring the USART
- 4 UART as an Instrument
- 5 Design Challenge

Serial versus Parallel

Serial

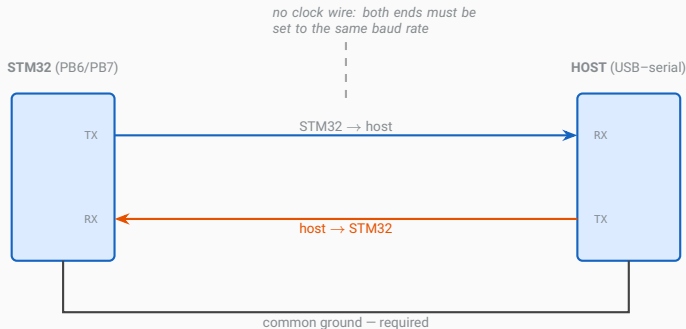
- One bit at a time, over a **single** wire
- Efficient over **long distances**
- Few pins

Parallel

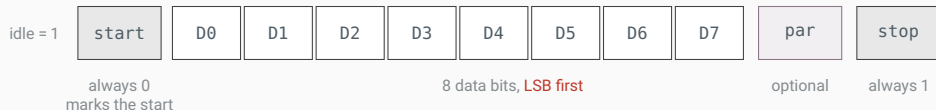
- Many bits at once, over **many** wires
- Faster over **short distances**
- Costs pins and board area



Asynchronous means no shared clock — so each byte carries its own framing. **Synchronous** adds a clock wire and removes the need for both ends to agree on speed in advance.



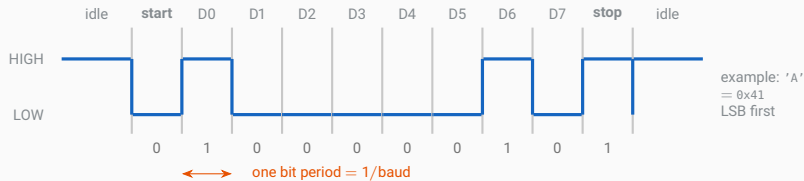
A UART link is three wires. Each device's **TX drives the other's RX** — the crossover — and both must share a **common ground**. There is no clock line, which is why both ends must independently agree on the baud rate.



Why asynchronous needs a start bit

With no shared clock, the receiver must know **when** a byte begins. The line drops from idle-high to low, and the receiver counts bit periods from that edge.

Both ends must agree in advance on **baud rate, data bits, parity and stop bits** — which is why “9600 8N1” is written down and quoted.



The falling edge of the **start bit** is the receiver's only timing reference. Data bits follow **least significant bit first**.

From here on: **USART**, not UART — that is the name of the actual peripheral on the STM32F446RE.

The u stands for *Universal*

The same hardware can also run **synchronous**, with an extra clock pin most designs leave unconnected. Used without that pin, a USART behaves exactly like a plain UART — which is how every example here uses it.

Same wire protocol, same frame, same registers below. USART is just the more general peripheral that happens to include UART mode.

- 1 Why This Matters
- 2 Serial Fundamentals
- 3 **Configuring the USART**
- 4 UART as an Instrument
- 5 Design Challenge

Register	Bit	Name	Function
USART_CR1	13	UE	USART enable
	12	M	Word length: 0 = 8 data bits, 1 = 9
	10	PCE	Parity control enable
	5	RXNEIE	Receive-interrupt enable
	3	TE	Transmitter enable
	2	RE	Receiver enable
USART_CR2	13:12	STOP	Stop bits: 00 = 1, 10 = 2 (default: 1)
USART_SR	7	TXE	Transmit register Empty – ready for next byte
	6	TC	Transmission complete – last bit has left the pin
	5	RXNE	Receive register Not Empty – a byte has arrived
USART_DR	7:0 (or 8:0)		Data – write to send, read to receive
USART_BRR	15:0		Baud rate divisor: mantissa in 15:4, fraction in 3:0

On the Nucleo, USART2 is wired to the ST-LINK and appears on your PC as a **virtual COM port**.

$$\text{USARTDIV} = \frac{f_{\text{CK}}}{16 \times \text{baud}}$$

The result goes in BRR, with the fractional part in the low four bits.

Baud	f_{CK}	BRR
9600	16 MHz	0x0683
115200	16 MHz	0x008B

Check: $16,000,000 / (16 \times 9600) = 104.17$. The integer 104 is 0x68, and $0.17 \times 16 \approx 3$ — giving **0x683**.

A mismatched baud rate is the classic first fault

Characters arrive, but as nonsense. If the very first thing you ever receive is garbage, suspect **baud rate** before anything else.

Configuring USART2

```
void USART2_init(void) {
    // 1. Clocks: GPIOA on AHB1,
    //    USART2 on APB1
    RCC->AHB1ENR |= (1U << 0);
    RCC->APB1ENR |= (1U << 17);

    // 2. PA2/PA3 to alt. function (10)
    GPIOA->MODER &= ~(3U<<(2*2)) | (3U<<(3*2));
    GPIOA->MODER |= ((2U<<(2*2)) | (2U<<(3*2)));

    // 3. AF7 (USART2), AFR[0]
    GPIOA->AFR[0] &= ~(0xFU<<(2*4)) | (0xFU<<(3*4));
    GPIOA->AFR[0] |= ((7U<<(2*4)) | (7U<<(3*4)));

    // 4. Baud: 9600 @ 16 MHz
    USART2->BRR = (104U << 4) | 3U;

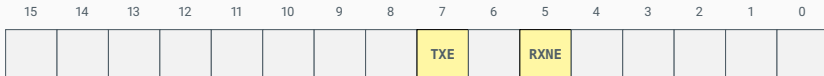
    // 5. 8N1, enable TX and RX
    USART2->CR1 = (1U<<3) | (1U<<2);

    // 6. Enable peripheral last
    USART2->CR1 |= (1U << 13);
}
```

Three peripheral blocks, in order: **clocks**, then **pins**, then the **USART** itself. **UE** always comes last — the peripheral ignores configuration changes once it is enabled.

This is the function every program from here on calls once, at the start of `main()`.

TXE and RXNE: The Two Status Bits



Bit	Meaning	Used by
SR bit 7, TXE	Transmit data register empty – safe to write DR	Program 1, every send
SR bit 5, RXNE	Receive data register not empty – a byte is waiting	Program 2, every receive

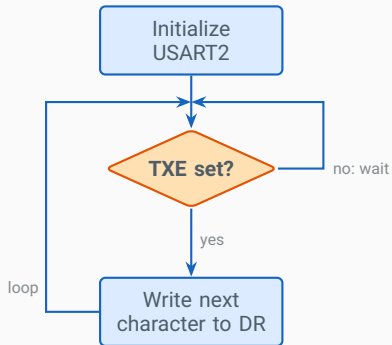
Same shape, opposite direction

Both programs are the same three-step pattern: **initialize**, then **wait** on one status bit, then **act** in the loop body. TXE waits for room to send; RXNE waits for something to arrive. Everything else in this chapter builds on this shape.

Program 1: Sending “Hello World!”

```
void USART2_write(int c)
{
    while (!(USART2->SR
        & (1U << 7))) ; // wait TXE
    USART2->DR = (c & 0xFF);
}

int main(void)
{
    USART2_init();
    const char *msg = "Hello World!\r\n";
    while (*msg) {
        USART2_write(*msg++);
    }
    while (1) { }
}
```

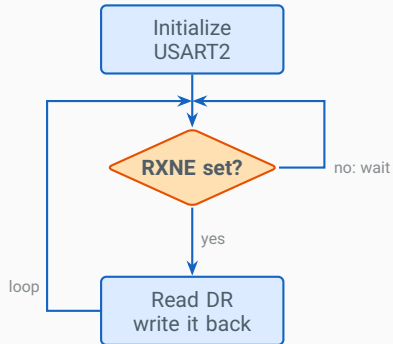


Each character waits for TXE (bit 7 of SR) before the next write – one wait loop per byte.

Program 2: Displaying Received Characters

```
int USART2_read(void)
{
    while (!(USART2->SR
        & (1U << 5))) ; // wait RXNE
    return USART2->DR;
}

int main(void)
{
    USART2_init();
    while (1) {
        int c = USART2_read();
        USART2_write(c); // show it
    }
}
```



Reading DR clears **RXNE** (bit 5 of SR) automatically — the hardware never needs to be told explicitly.

- 1 Why This Matters
- 2 Serial Fundamentals
- 3 Configuring the USART
- 4 UART as an Instrument**
- 5 Design Challenge

What it buys you

- See variables **while the program runs**, without halting it
- Timestamp events and measure intervals
- Log a fault that happens once an hour

What it costs

At 9600 baud, one character takes about **1 ms**. A 40-character line is **40 ms** — which can be longer than the deadline you are trying to debug.

The observer effect is real

Adding `printf` to a timing-sensitive routine can make the bug disappear — or create a new one. Use a **higher baud rate**, **shorter messages**, or set a **GPIO pin** and watch it on a scope instead.

Printing Numbers Over UART

```
// Decimal, without printf
void uart_print_uint(uint32_t v)
{
    char buf[11]; int i = 0;
    if (v == 0) {
        uart_write('0'); return;
    }
    while (v > 0) {          // LSD first
        buf[i++] = '0' + (v % 10);
        v /= 10;
    }
    while (i > 0) {          // reverse
        uart_write((uint8_t)buf[--i]);
    }
}

// Hex -- ideal for registers
void uart_print_hex(uint32_t v)
{
    const char *d = "0123456789ABCDEF";
    uart_print("0x");
    for (int s = 28; s >= 0; s -= 4) {
        uart_write((uint8_t)d[(v>>s)&0xF]);
    }
}
```

A few hundred bytes buy a diagnostic channel that works in every program from here on:

```
uart_print("MODER = ");
uart_print_hex(GPIOA->MODER);
uart_print("\r\n");
```

Avoid printf: it pulls in tens of kilobytes and may need the heap. These cost almost nothing.

Interrupt-Driven Reception

```
volatile char    rx_buf[64];
volatile uint8_t rx_head = 0;

void USART2_IRQHandler(void)
{
    if (USART2->SR & (1U << 5)) {           // RXNE
        rx_buf[rx_head++ & 63] = USART2->DR; // reading DR clears RXNE
    }
}
```

Why polling a UART does not scale

At 115200 baud a byte arrives every **87 μ s**. If your main loop takes longer than that between polls, the **overrun** flag sets and data is *lost* — silently.

An interrupt collects each byte the moment it arrives, and `main()` processes the buffer when convenient.

A Second Serial Link: When USART2 Is Not Enough

Every example so far uses USART2, because the Nucleo's built-in ST-LINK makes it the easiest way to talk to a PC. But that link is also its limit: it is already committed. The first time a design needs a **second** serial device — a GPS module, a Bluetooth radio — while *still* printing debug output, one USART is no longer enough.

```
void USART1_init(void)
{
    RCC->AHB1ENR |= (1U << 1); // GPIOB
    RCC->APB2ENR |= (1U << 4); // USART1
                                // -- APB2, not APB1

    // ... same GPIO/AF7 steps,
    // on PB6/PB7 instead of PA2/PA3
    USART1->BRR = 0x0683;
    USART1->CR1 = (1U << 3) | (1U << 2);
    USART1->CR1 |= (1U << 13);
}
```

USART1, on PB6/PB7, is the natural second choice: same AF7 selection, same init shape. The only thing that matters conceptually is that USART1 sits on **APB2**, while USART2 sits on **APB1** — the same f_{CK} question from the baud rate section, now relevant because the two peripherals no longer share a clock source.

- **Serial** sends one bit at a time — fewer pins, longer distance. **Parallel** is faster but short-range.
- **UART is asynchronous**: no shared clock, so each byte is framed with a **start bit**, data (**LSB first**), optional parity, and a **stop bit**.
- Both ends must agree on **baud, data bits, parity, stop** — “9600 8N1”.
- $USARTDIV = f_{CK} / (16 \times \text{baud})$, written to BRR.
- Transmit: wait **TXE**, write DR. Receive: wait **RXNE**, read DR.
- UART is your best **diagnostic instrument** — but printing takes real time, and can hide or create timing bugs.

Next: Chapter 13 — SPI and I²C

Adding a clock wire removes the speed ceiling.

- 1 Why This Matters
- 2 Serial Fundamentals
- 3 Configuring the USART
- 4 UART as an Instrument
- 5 **Design Challenge**

The brief

Design a diagnostic protocol for the sensor monitor, to be carried over UART at 115200 baud, so that a host can observe the device without disturbing its timing.

Specify the message format, what is reported and how often, and how the host distinguishes a measurement from an error report. Compute the total bandwidth your protocol consumes and confirm it fits within the CPU budget. Then state which single piece of diagnostic information would be most useful to a field technician, and why you chose it over the alternatives.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 12